Национална програма
"Обучение за ИТ умения и кариера"
https://it-kariera.mon.bg

Министерството на
образованието и науката
https://www.mon.bg

ASP.NET MVC като потребителски интерфейс

Разработка на софтуер

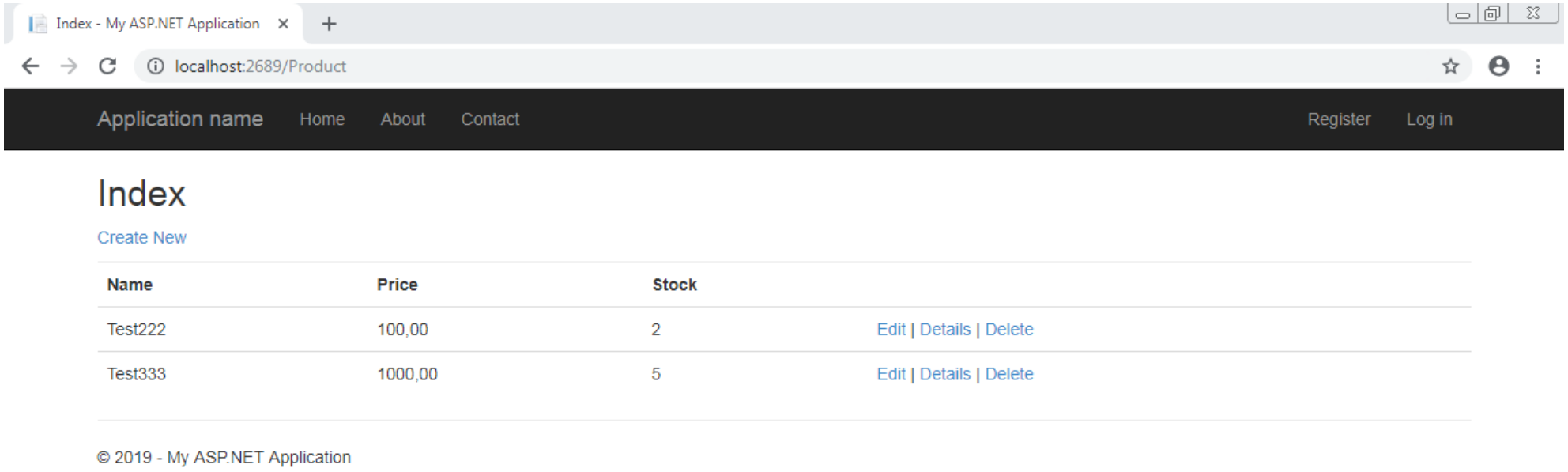
Съдържание

- Използване на ASP.NET MVC за презента-ционен слой
- Разделяне на приложението на проекти според слоя

Какво е ASP.NET MVC

- ASP.NET MVC е рамка за изграждане на уеб приложения.
  - Базирана на MVC шаблона
  - Лесна и бърза за работа
  - Чудесен визуален интерфейс за нашето приложение

  Благодарение на ASP.NET MVC ние ще дадем добър и удобен външен вид на нашето CRUD приложение

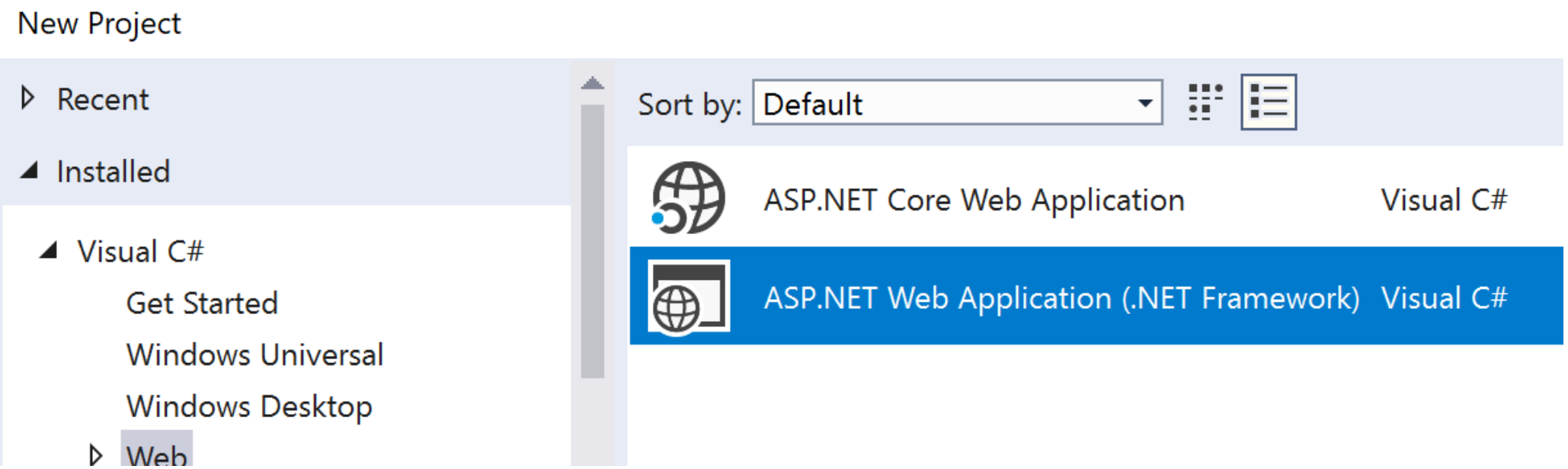


Структура на проекта ни

С участието на ASP.NET MVC

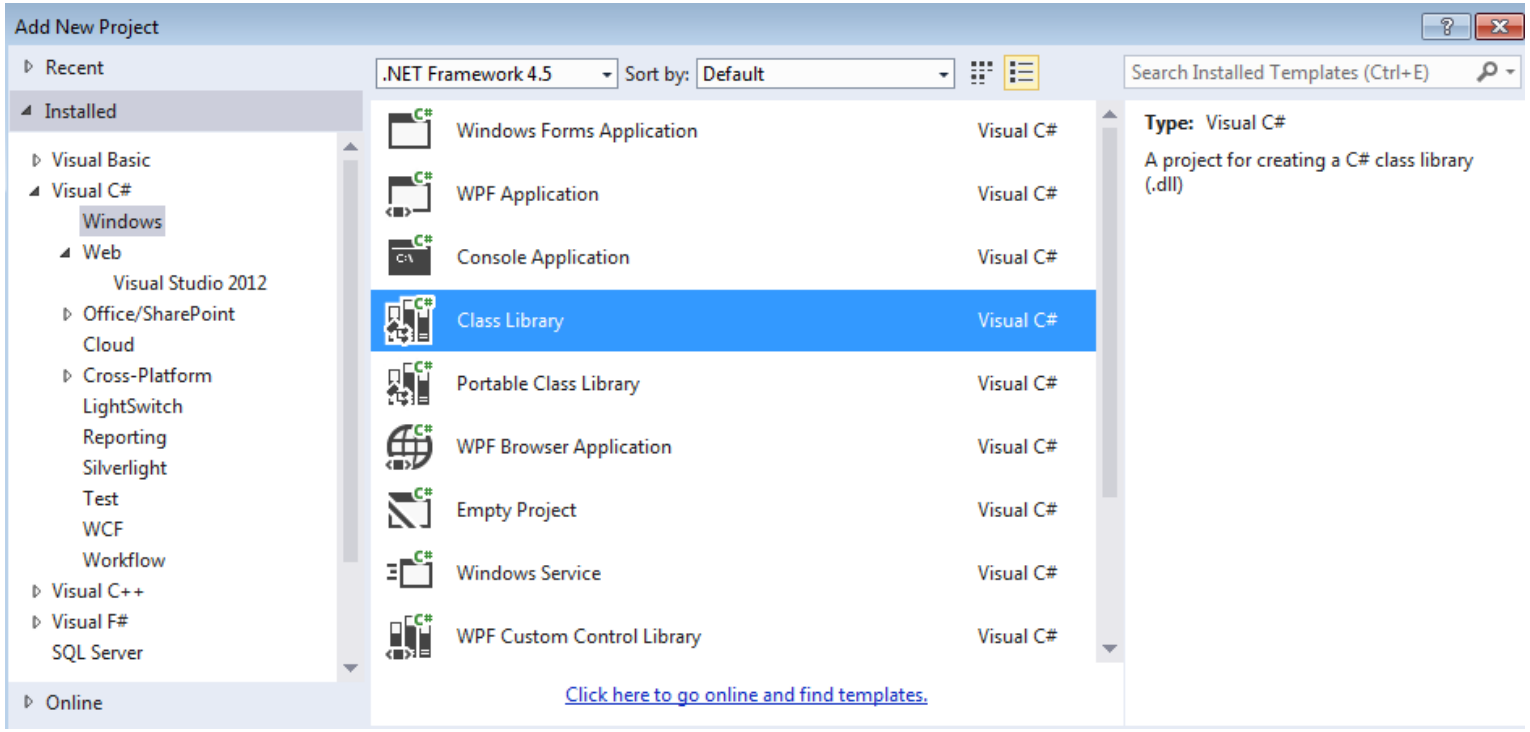
Какви проекти трябва да имаме? [1/3]

- ASP.NET Web Application
  - Това ще е проекта ни, който ще действа като презентационен слой



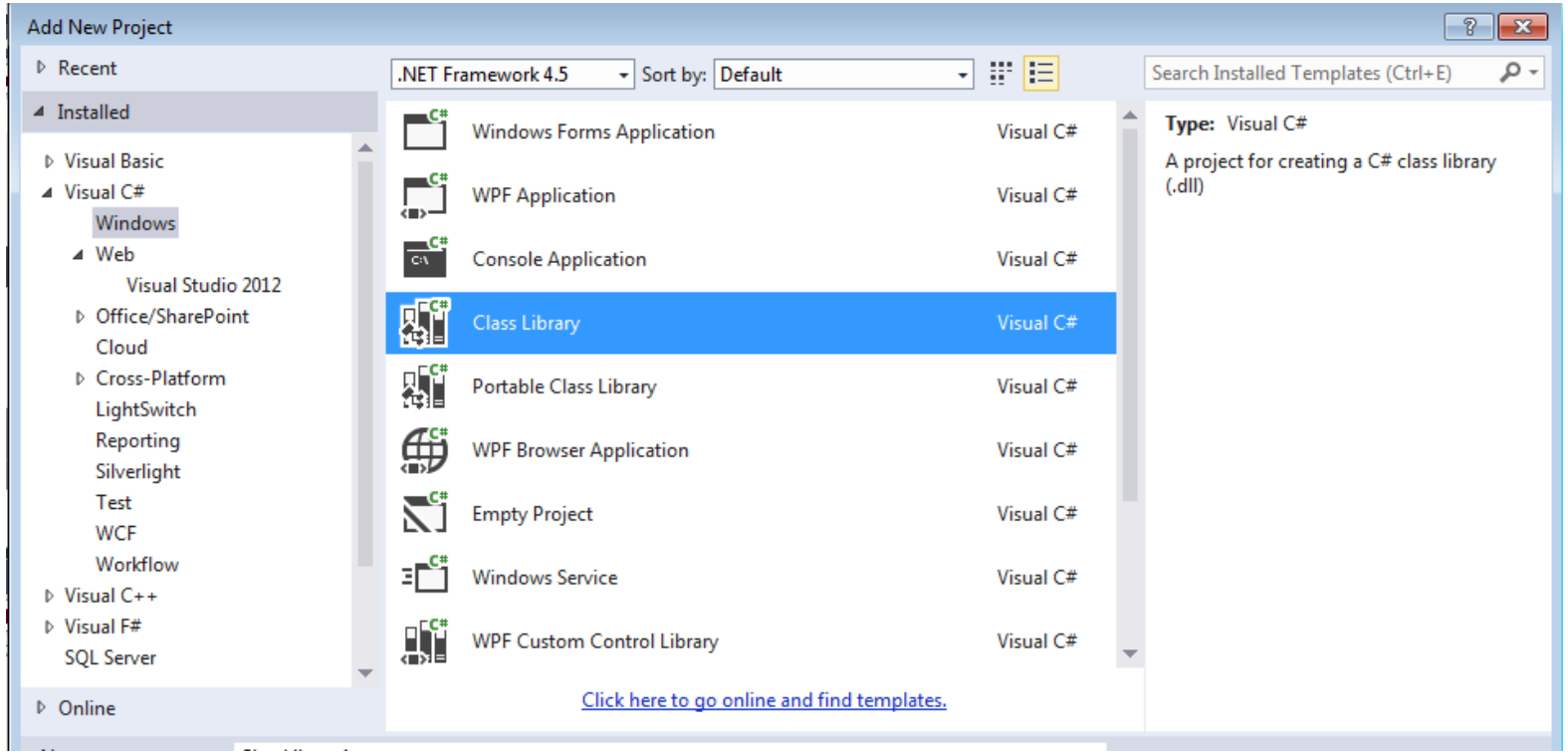
Какви проекти трябва да имаме? [2/3]

- Data - това ще е проекта ни, който ще бъде слоят за данни. При създаването му от Visual C# -> Windows трябва да изберем Class library



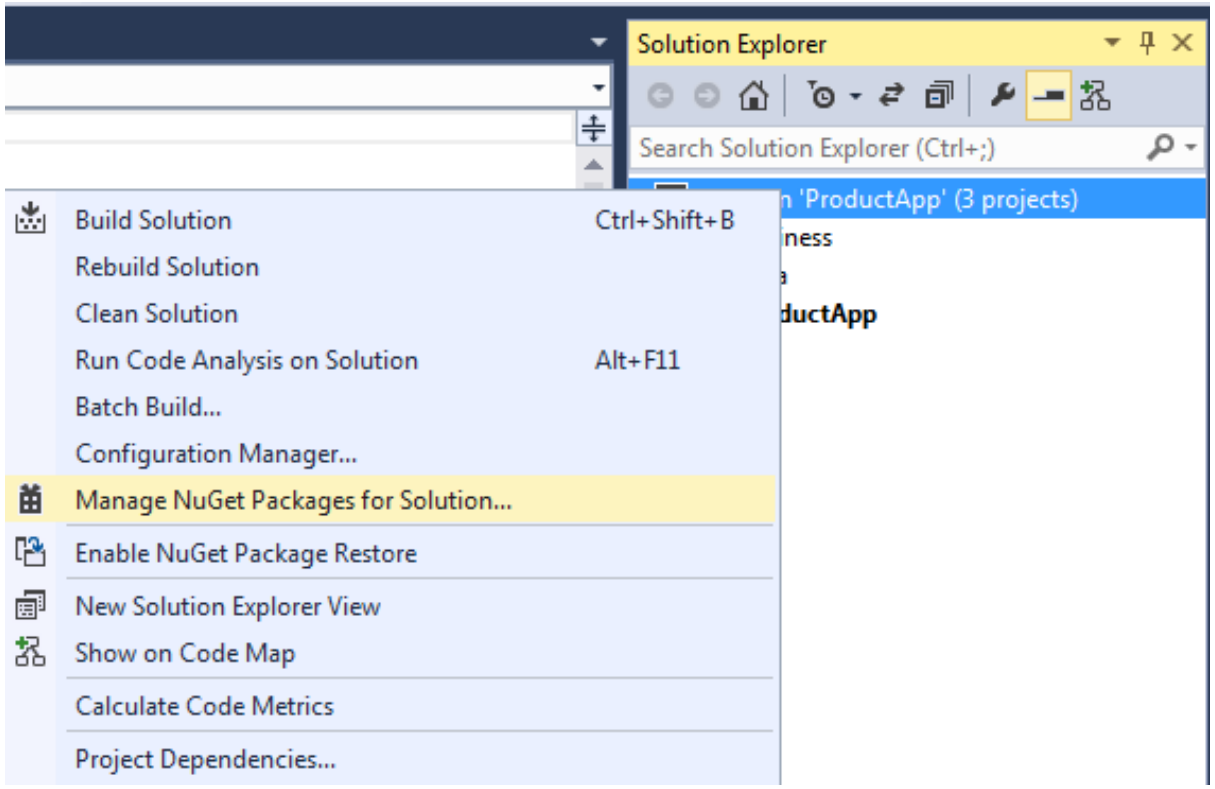
Какви проекти трябва да имаме? [3/3]

- Business - това ще е проекта ни, който ще бъде слоят за услуги. При създаването му от Visual C# -> Windows трябва да изберем Class library



Добавяне на EntityFramework

- Цъкнете с десен бутон върху solution-а и изберете Manage NuGet Packages for Solution и инсталирайте EntityFramework. Изберете инсталация
за всичките проекти
в solution-а



Добавяне на референции

- На проект Business изберете [Add] -> References. Добавете референция към Data
- На проект ProductApp /проектът на ASP.NET/ изберете [Add] -> References. Добавете референция към Business и Data

Data/Model/Product.cs

- Създайте подпапка Model в Data, а в нея модел клас Product.cs

```
public class Product
{
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public int Stock { get; set; }
}
```

Data/ProductContext.cs

- Създайте ProductContext клас, който наследява DbContext. Ще има нужда да добавите и using директива:

```
using System.Data.Entity;

public class ProductContext : DbContext
{
        public ProductContext() : base("name=ProductContext")
        {

        }
        public DbSet<Product> Products { get; set; }
}
```

Business/ProductBusiness.cs

- Създаваме управляващо поле от клас ProductContext:

```
private ProductContext context;
```

- Създаваме методи, които извършват желаните операции, извиквайки свойството Products и методи върху него, познати ни от LINQ:

```
public List<Product> GetAll() {
    using (productContext = new ProductContext()) {
        return productContext.Products.ToList();
    }
}
```

- По аналогичен начин реализираме другите методи

Свързване на ASP.NET с другите слоеве [1/2]

- В Web.config намираме <connectionStrings> секцията. Там под другия <add> добавяме и нашия:

```
<add name="ProductContext"
connectionString="Data Source=.; Initial
Catalog=ProductDb; Integrated Security=true„
providerName="System.Data.SqlClient" />
```

- След тази стъпка трябва да се уверим, че сме направили Build на целия Solution

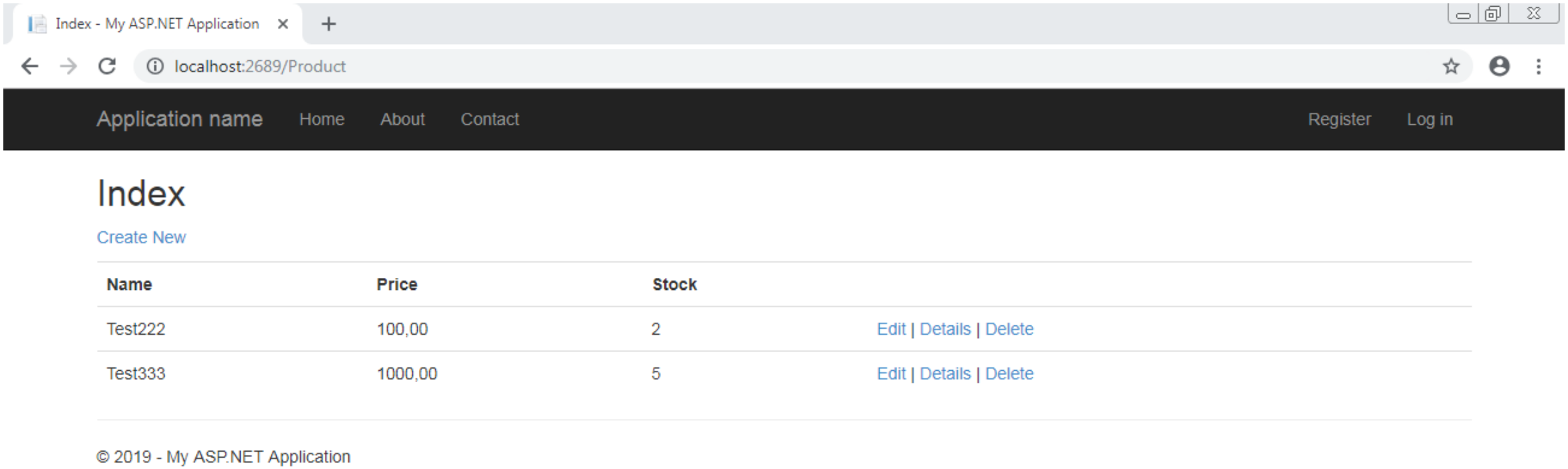
Свързване на ASP.NET с другите слоеве [1/2]

- Добавяме контролер, чрез десен бутон върху Controllers папката -> Add -> Controller
- Изберете MVC Controller with views
- За име на контролера задайте ProductController
- За Model клас посочете Product (Data.Model)
- За Data Context клас посочете ProductContext (Data)

Използване на приложението [1/2]

- Компилирайте и стартирайте приложението.
- Добавете /Product в края на URL адреса и заредете страницата
- Пред вас вече е наличен уеб интерфейса за нашето приложение, в който може да извършите всички CRUD операции

Използване на приложението [2/2]



Национална програма
"Обучение за ИТ умения и кариера"
https://it-kariera.mon.bg

Министерството на
образованието и науката
https://www.mon.bg

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).